Universidade Federal de Uberlândia
Engenharia de Computação · Banco de Dados

TRABALHO PRÁTICO

Sistema de Gestão de Clínica Veterinária

Modelagem conceitual, lógica e física e implementação de um banco de dados relacional em PostgreSQL

Aluno	Samuell — matrícula 12511ECP003
SGBD	PostgreSQL 16 · pgAdmin 4
Entregáveis	este relatório · `sql/00_criar_banco.sql` · `01_ddl.sql` · `02_dml.sql` · `03_consultas.sql`

Uberlândia — outubro de 2026

Sumário

1. Descrição do sistema 3
2. Modelo conceitual 4
3. Modelo lógico 8
4. Modelo físico 12
5. Scripts DDL e DML 17
6. Consultas obrigatórias 19
7. Consultas complementares 25
8. Interface web 29
9. Considerações finais 31
Apêndice A — Como executar 32

2. Descrição do sistema

O sistema controla a rotina clínica e administrativa de uma clínica veterinária de pequeno porte: o cadastro de **tutores** e de seus **animais**, o corpo clínico de **veterinários**, a **agenda** de horários, os **atendimentos** realizados e, dentro de cada atendimento, os **procedimentos** executados e os **medicamentos prescritos**. O conjunto dos atendimentos de um animal forma o seu **histórico clínico** (prontuário).

O fluxo típico é: o tutor é cadastrado com seus animais; a recepção marca um horário com um veterinário; no dia, o atendimento é aberto a partir do agendamento (ou sem agendamento, em urgências); o veterinário registra queixa, exame, diagnóstico, procedimentos e prescrições; ao final, o atendimento é finalizado. Atendimentos que se estendem, como internações, ficam *em aberto* até a alta.

2.1 Regras de negócio

Cód.	Regra	Como é garantida no banco
RN01	Um animal pertence a **um, e apenas um**, tutor. Um tutor pode ter vários animais.	FK `animal.id_tutor` `NOT NULL`
RN02	Todo animal tem uma raça, e toda raça pertence a uma espécie. Animais sem raça definida usam a raça "SRD" da sua espécie.	FKs `NOT NULL`; `UNIQUE (id_especie, nome)` em `raca`
RN03	Um atendimento é realizado por **um** veterinário e gera **zero ou mais** procedimentos e prescrições.	FK `NOT NULL`; tabelas associativas
RN04	O agendamento tem status `AGENDADO`, `REALIZADO` ou `CANCELADO`; cancelar exige motivo.	`CHECK` de domínio e `CHECK` condicional
RN05	Um veterinário não pode ter dois horários ativos sobrepostos.	Restrição de exclusão GiST
RN06	Um agendamento gera no máximo um atendimento, do **mesmo animal**; ao gerar, passa a `REALIZADO`. Agendamento cancelado não gera atendimento.	`UNIQUE` + FK composta + gatilhos
RN07	Atendimento é `EM_ABERTO` ou `FINALIZADO`; só o finalizado tem data/hora de término.	`CHECK` de coerência
RN08	O valor cobrado de um procedimento é o da data do atendimento; reajustes de tabela não alteram o histórico.	Coluna `valor_unitario` + gatilho de preço padrão
RN09	Animais com óbito registrado não recebem novos atendimentos; o histórico é preservado.	Gatilho + `ON DELETE RESTRICT`
RN10	Profissionais desligados e procedimentos descontinuados são inativados, nunca apagados.	Coluna `ativo`; FKs `RESTRICT`
RN11	O CPF de tutores e veterinários precisa ser válido: dígitos verificadores corretos e não pode ser uma sequência repetida.	Função `cpf_valido()` no `CHECK`
RN12	Procedimentos e medicamentos podem ser restritos a certas espécies (V10 só em cão, Simparic não vai para gato). Itens sem restrição valem para todas.	Tabelas `procedimento_especie` e `medicamento_especie` + gatilho

3. Modelo conceitual

O Diagrama Entidade-Relacionamento (Figura 1) usa a notação de Chen: retângulos são entidades (com seus atributos e o identificador sublinhado), losangos são relacionamentos e as notas cinzas trazem atributos dos relacionamentos N:N. A cardinalidade **(mín, máx)** escrita junto a uma entidade indica quantas ocorrências *dela* se associam a uma ocorrência da entidade do outro lado. Mínimo 0 significa participação parcial; mínimo 1, participação total.

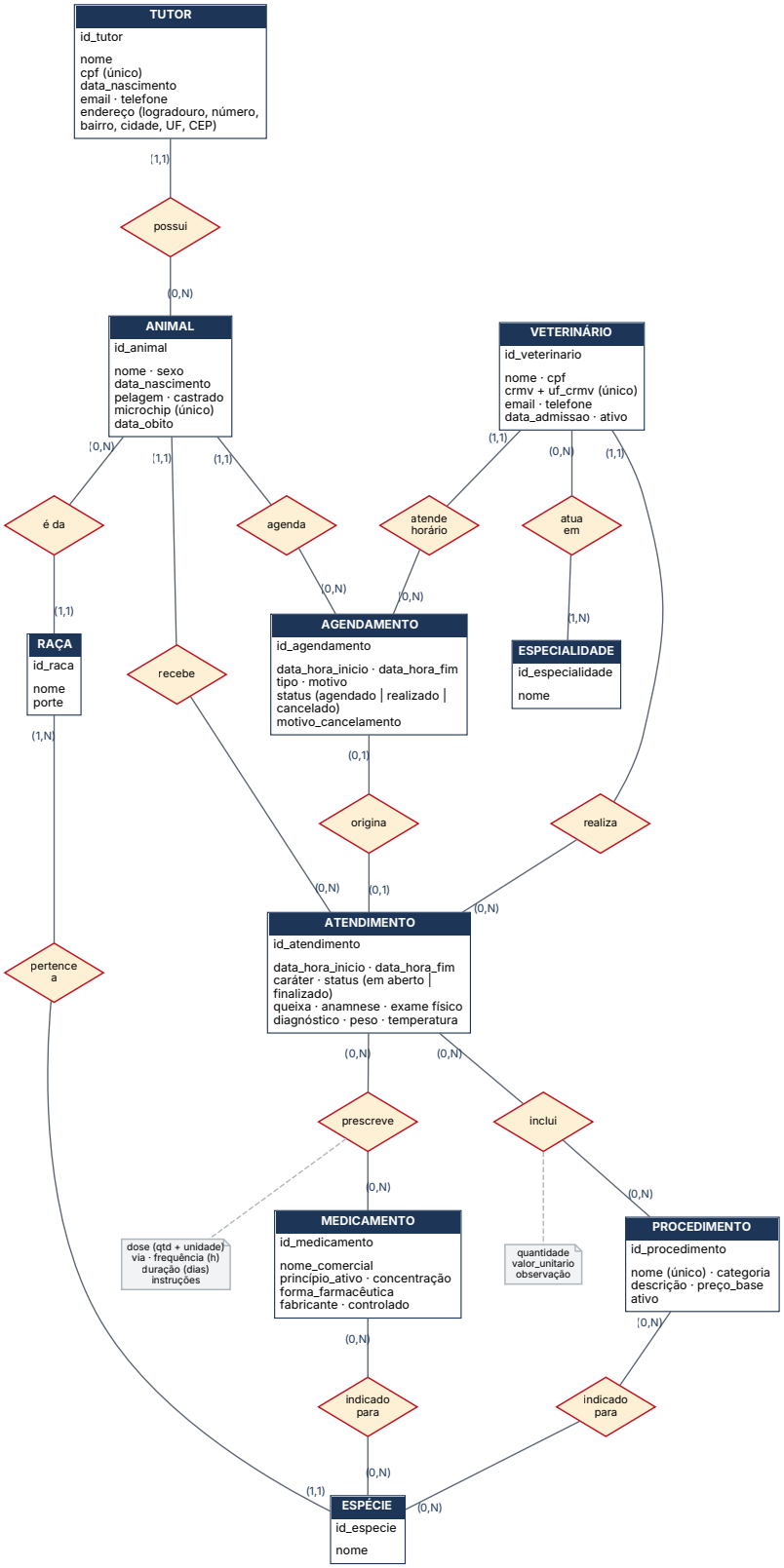


Figura 1. DER conceitual. Leitura de exemplo: um TUTOR possui (0,N) ANIMAIS; um ANIMAL pertence a (1,1) TUTOR.

3.1 Relacionamentos, cardinalidades e participação

Relacionamento	Entidades	Card.	Participação	Leitura
possui	Tutor — Animal	1:N	Animal total · Tutor parcial	Todo animal tem exatamente um tutor; um tutor pode estar cadastrado ainda sem animais.
é da	Animal — Raça	N:1	Animal total · Raça parcial	Todo animal tem uma raça; podem existir raças sem animais.
pertence a	Raça — Espécie	N:1	Ambas totais	Toda raça é de uma espécie; toda espécie tem ao menos a raça SRD.
atua em	Veterinário — Especialidade	N:N	Veterinário total · Especialidade parcial	Cada veterinário tem uma ou mais especialidades.
agenda	Animal — Agendamento	1:N	Agendamento total	Cada horário é de um animal.
atende horário	Veterinário — Agendamento	1:N	Agendamento total	Cada horário é com um veterinário.
origina	Agendamento — Atendimento	1:1	Ambas parciais	Um horário gera no máximo um atendimento; urgências não têm horário.
recebe	Animal — Atendimento	1:N	Atendimento total	O histórico clínico do animal é o conjunto dos seus atendimentos.
realiza	Veterinário — Atendimento	1:N	Atendimento total	Cada atendimento é responsabilidade de um veterinário (RN03).
inclui	Atendimento — Procedimento	N:N	Ambas parciais	Atributos: quantidade, valor unitário, observação.
prescreve	Atendimento — Medicamento	N:N	Ambas parciais	Atributos: dose, via, frequência, duração, instruções.
indicado para	Procedimento — Espécie	N:N	Ambas parciais	Procedimento sem espécies associadas vale para todas (RN12).
indicado para	Medicamento — Espécie	N:N	Ambas parciais	Mesma regra para medicamentos (RN12).

3.2 Decisões de modelagem

- **Espécie e Raça como entidades**, e não como texto livre no animal. Isso evita grafias diferentes para a mesma raça ("Shih-tzu", "shih tzu") e permite agrupar por espécie, que a consulta Q4 exige.
- **Procedimento e Medicamento são catálogos**. O que acontece em cada atendimento é registrado nos relacionamentos *inclui* e *prescreve*, que carregam os atributos do evento (quanto foi cobrado, qual dose foi receitada).
- **Agendamento e Atendimento são entidades distintas**. A agenda é uma intenção (pode ser cancelada ou não ter comparecimento); o atendimento é o fato clínico. Separá-los permite atendimentos sem horário (urgências) e cancelamentos sem apagar dados.
- **Histórico clínico não é uma entidade**: ele é derivado dos atendimentos do animal, com seus procedimentos e prescrições. No banco, isso vira a visão `vw_historico_clinico`.

- **Restrição por espécie como relacionamento**, e não como coluna. Um procedimento pode servir a várias espécies (castração: cão, gato e coelho), então a relação é N:N. A ausência de linhas significa “sem restrição”, o que evita cadastrar consulta e hemograma para todas as espécies.
- **Peso e temperatura ficam no atendimento**, não no animal, porque mudam ao longo do tempo e o histórico deles tem valor clínico.

4. Modelo lógico

O mapeamento do DER para o modelo relacional seguiu as regras usuais: cada entidade virou uma relação com chave primária própria; relacionamentos 1:N viraram chave estrangeira no lado N; o 1:1 parcial *origina* virou FK opcional e única em `atendimento`; e os N:N viraram relações associativas. Abaixo, o esquema em notação textual (chave primária sublinhada, chave estrangeira em *itálico*, chave candidata marcada com ᵁ).

```
ESPECIE (id_especie, nomeᵁ)

RACA (id_raca, id_especie → ESPECIE, nome, porte) — (id_especie, nome)ᵁ

TUTOR (id_tutor, nome, cpfᵁ, data_nascimento, emailᵁ, telefone, logradouro, numero, complemento, bairro, cidade, uf, cep, criado_em)

ANIMAL (id_animal, id_tutor → TUTOR, id_raca → RACA, nome, sexo, data_nascimento, pelagem, castrado, microchipᵁ, data_obito, observacoes, criado_em)

VETERINARIO (id_veterinario, nome, cpfᵁ, crmv, uf_crmv, emailᵁ, telefone, data_admissao, ativo) — (uf_crmv, crmv)ᵁ

ESPECIALIDADE (id_especialidade, nomeᵁ)

VETERINARIO_ESPECIALIDADE (id_veterinario → VETERINARIO, id_especialidade → ESPECIALIDADE)

AGENDAMENTO (id_agendamento, id_animal → ANIMAL, id_veterinario → VETERINARIO, data_hora_inicio, data_hora_fim, tipo, motivo, status, motivo_cancelamento, criado_em)

ATENDIMENTO (id_atendimento, id_animal → ANIMAL, id_veterinario → VETERINARIO, id_agendamentoᵁ, data_hora_inicio, data_hora_fim, carater, status, queixa_principal, anamnese, exame_fisico, diagnostico, peso_kg, temperatura_c, observacoes) — (id_agendamento, id_animal) → AGENDAMENTO(id_agendamento, id_animal)

PROCEDIMENTO (id_procedimento, nomeᵁ, categoria, descricao, preco_base, ativo)

ATENDIMENTO_PROCEDIMENTO (id_atendimento → ATENDIMENTO, id_procedimento → PROCEDIMENTO, quantidade, valor_unitario, observacao)

PROCEDIMENTO_ESPECIE (id_procedimento → PROCEDIMENTO, id_especie → ESPECIE)

MEDICAMENTO_ESPECIE (id_medicamento → MEDICAMENTO, id_especie → ESPECIE)

MEDICAMENTO (id_medicamento, nome_comercial, principio_ativo, concentracao, forma_farmaceutica, fabricante, controlado) — (nome_comercial, concentracao, forma_farmaceutica)ᵁ

PRESCRICAO (id_prescricao, id_atendimento → ATENDIMENTO, id_medicamento → MEDICAMENTO, dose_quantidade, dose_unidade, via_administracao, frequencia_horas, duracao_dias, instrucoes) — (id_atendimento, id_medicamento)ᵁ
```

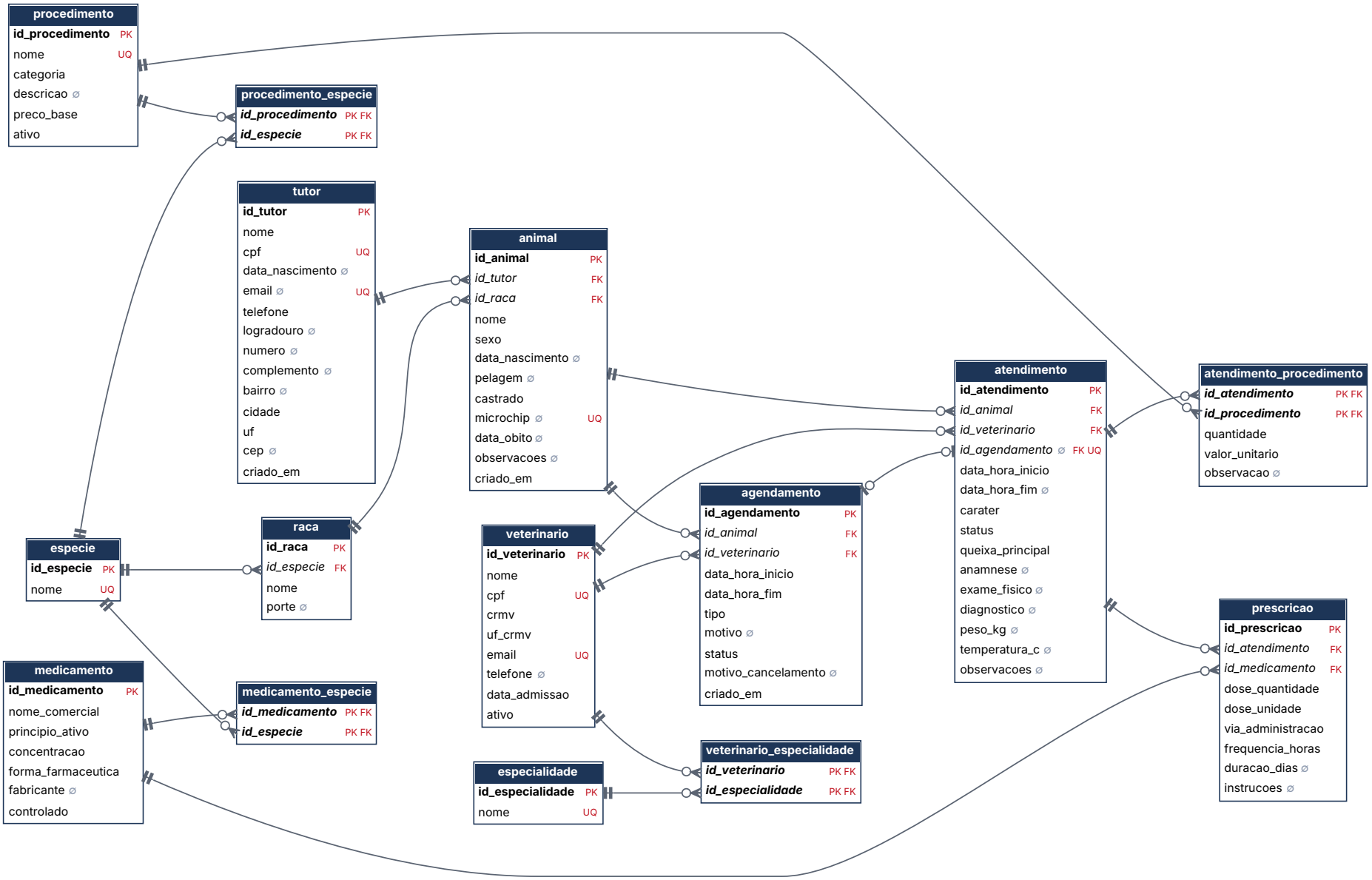


Figura 2. Modelo lógico gerado a partir do catálogo do próprio banco (`scripts/gerar_diagrama_logico.py`). Negrito = PK; itálico = FK; ø = aceita nulo; UQ = único. Notação pé-de-galinha: ‖ exatamente um, O| zero ou um, O< zero ou muitos.

4.1 Restrições de integridade

Tipo	Onde	Finalidade
Entidade (PK)	Todas as tabelas	Chave substituta `IDENTITY`; PK composta nas associativas puras (`veterinario_especialidade`, `atendimento_procedimento`).
Chaves candidatas	cpf, email, crmv+UF, microchip, nomes de catálogo	Impedem cadastros duplicados no mundo real mesmo com PK artificial.
Referencial (FK)	18 chaves estrangeiras	`RESTRICT` onde há histórico a preservar; `CASCADE` só em itens que não existem sem o pai (procedimentos e prescrições do atendimento, especialidades do veterinário, agenda do animal).
FK composta	`atendimento (id_agendamento, id_animal)`	Garante que o atendimento vinculado a um horário seja do mesmo animal do horário (RN06).
Domínio (CHECK)	status, tipo, caráter, categoria, sexo, porte, via, unidade, UF	Restringe colunas a valores válidos.
Formato (CHECK)	CPF, telefone, CEP, e-mail, microchip, CRMV	Expressões regulares: só dígitos e no tamanho certo. O CPF vai além do formato: a função `cpf_valido()` confere os dois dígitos verificadores (módulo 11).
Semântica (CHECK)	agendamento, atendimento, animal, prescrição	Fim depois do início; cancelado ⇔ tem motivo; finalizado ⇔ tem fim; peso > 0; temperatura entre 30 e 45 °C; óbito ≥ nascimento; duração do horário ≤ 8 h.
Exclusão (GiST)	`agendamento`	Mesmo veterinário + intervalos sobrepostos é proibido, ignorando cancelados (RN05).
Gatilhos	atendimento, agendamento, atendimento_procedimento	Regras que envolvem mais de uma tabela (RN06, RN08, RN09).

4.2 Justificativa da normalização

1ª Forma Normal. Todos os atributos são atômicos e não há grupos repetitivos. Os pontos que exigiram decisão foram:

- o *endereço* foi decomposto em logradouro, número, complemento, bairro, cidade, UF e CEP;
- a *dose* foi separada em quantidade numérica e unidade ("1,5" + "comprimido"), e a frequência é um inteiro em horas, em vez de um texto livre como "1,5 comp. de 12/12 h";
- atributos multivalorados (as especialidades de um veterinário, os procedimentos e as prescrições de um atendimento) viraram tabelas próprias em vez de listas numa coluna.

2ª Forma Normal. Quatro tabelas têm chave primária composta. Em `veterinario_especialidade`, `procedimento_especie` e `medicamento_especie` não há atributos não-chave. Em `atendimento_procedimento` (PK = id_atendimento + id_procedimento), `quantidade`, `valor_unitario` e `observacao` dependem da chave inteira. Em especial, `valor_unitario` **não** depende só do procedimento: é o preço praticado *naquele* atendimento (pode ter desconto ou retorno gratuito). O preço de tabela, que depende só do procedimento, está em `procedimento.preco_base`. As demais tabelas têm PK simples, logo estão trivialmente em 2FN.

3ª Forma Normal. Nenhum atributo não-chave depende de outro atributo não-chave:

- `animal` guarda apenas `id_raca`. Guardar também a espécie criaria a dependência transitiva id_animal → id_raca → id_especie; a espécie é obtida pela raça.

- Dados derivados não são armazenados: idade (calculada na visão `vw_animal`) e valor total do atendimento (soma dos itens).
- Em `atendimento`, poderia parecer que id_agendamento → id_animal é transitiva. Mas `id_agendamento` é opcional (urgências não têm horário), então ele não determina o animal em todas as tuplas e `id_animal` precisa existir por si. A coerência entre os dois é garantida pela FK composta. O veterinário do atendimento também não é derivável do agendamento, porque outro profissional pode cobrir o horário.
- **Endereço:** CEP → logradouro/bairro não é uma dependência funcional no Brasil, pois muitas cidades têm um CEP único para todas as ruas (por exemplo, 38440-000 em Araguari, presente nos dados). Do mesmo modo, cidade ↛ UF, porque há municípios homônimos em estados diferentes. Logo, não há transitividade a remover.

Em todas as tabelas, os únicos determinantes são chaves candidatas, de modo que o esquema também atende à **Forma Normal de Boyce-Codd**.

> **Redundância controlada.** O status `REALIZADO` do agendamento pode ser deduzido da existência de um atendimento vinculado. Ele foi mantido de propósito porque a agenda é consultada o tempo todo e filtrar por uma coluna é mais simples e barato que fazer um anti-join. Para não haver divergência, o valor é mantido por gatilho (seção 4.4), e não pela aplicação.

5. Modelo físico

O SGBD é o **PostgreSQL**, como recomendado. Além de ser livre e padrão de mercado, ele oferece recursos que este projeto usa diretamente: `TIMESTAMPTZ`, colunas `IDENTITY` do padrão SQL, índices parciais e de expressão, restrições de exclusão com GiST, gatilhos em PL/pgSQL e comentários de catálogo. Todos os objetos ficam no esquema `clinica`, separado do `public`.

5.1 Tipos de dados

Dado	Tipo	Justificativa
Chaves primárias	`INTEGER` / `SMALLINT` `GENERATED BY DEFAULT AS IDENTITY`	IDENTITY é o padrão SQL e substitui o `SERIAL` legado: a sequência pertence à coluna. `SMALLINT` (2 bytes) nos catálogos que nunca passarão de dezenas de linhas (espécie, especialidade); `INTEGER` (4 bytes, até 2,1 bi) nas demais.
CPF, CEP, UF, microchip	`CHAR(11)`, `CHAR(8)`, `CHAR(2)`, `CHAR(15)`	Tamanho fixo por definição. São armazenados como texto, não como número, porque podem começar com zero e nunca entram em contas. O formato é validado por `CHECK` com expressão regular.
Telefone	`VARCHAR(11)`	10 ou 11 dígitos (fixo ou celular com DDD), sem máscara. A formatação fica na apresentação.
Nomes, e-mail, motivos	`VARCHAR(n)`	Limite explícito documenta o tamanho esperado e impede lixo. No PostgreSQL, `VARCHAR(n)` não ocupa mais espaço que o conteúdo.
Anamnese, diagnóstico, instruções	`TEXT`	Texto clínico livre, sem limite natural de tamanho.
Valores monetários	`NUMERIC(10,2)`	Aritmética decimal exata. `REAL` / `DOUBLE` causariam erros de arredondamento em somas de faturamento; o tipo `MONEY` depende do locale do servidor.
Peso	`NUMERIC(6,3)`	Três casas decimais para pesar aves e roedores (o periquito dos dados pesa 0,035 kg); até 999,999 kg cobre cães gigantes.
Temperatura	`NUMERIC(3,1)`	Uma casa decimal, faixa validada entre 30 e 45 °C.
Dose	`NUMERIC(8,3)` + `VARCHAR(10)`	Doses fracionadas (0,25 comprimido; 0,4 mL) com unidade controlada por `CHECK`.
Data e hora de eventos	`TIMESTAMPTZ`	Guarda o instante absoluto (UTC) e exibe no fuso da sessão. Evita ambiguidade em mudanças de fuso ou horário de verão. O banco é configurado com `timezone = America/Sao_Paulo`.
Nascimento, óbito, admissão	`DATE`	Hora não é relevante.
Indicadores	`BOOLEAN`	`castrado`, `ativo`, `controlado`, com `DEFAULT` definido.
Status, tipo, categoria	`VARCHAR` + `CHECK (... IN ...)`	Preferido a `ENUM` porque incluir ou remover um valor é só trocar a restrição. Com `ENUM`, remover um valor exige recriar o tipo. Também é mais portável.

5.2 Índices

O PostgreSQL cria índices B-tree automaticamente para PKs e `UNIQUE`, mas **não** para chaves estrangeiras. Os índices abaixo foram criados para cobrir FKs usadas em junções e os filtros das consultas da seção 6. No total, o esquema tem 43 índices, contando os automáticos.

Índice	Colunas / tipo	Justificativa
`ix_animal_tutor`	animal (id_tutor)	"Animais do tutor" (Q3) e verificação da FK ao remover tutores.
`ix_animal_raca`	animal (id_raca)	Junção animal → raça → espécie (Q4, visões).
`ix_agendamento_animal`	agendamento (id_animal)	Agenda de um animal; suporte ao `ON DELETE CASCADE`.
`ix_atendimento_animal_data`	atendimento (id_animal, data_hora_inicio DESC)	Composto: monta o histórico clínico já na ordem "mais recente primeiro", sem etapa de ordenação. Também cobre a FK `id_animal`.
`ix_atendimento_vet_data`	atendimento (id_veterinario, data_hora_inicio)	Composto: o ranking por período (Q2) filtra por veterinário e faixa de datas. A igualdade vem primeiro e o intervalo depois, que é a ordem que o B-tree aproveita.
`ix_agendamento_futuro`	agendamento (data_hora_inicio) WHERE status = 'AGENDADO'	**Parcial**: só indexa horários pendentes, que são poucos perto do histórico de realizados e cancelados. Atende Q1 e a tela de agenda.
`ix_atendimento_em_aberto`	atendimento (id_animal) WHERE status = 'EM_ABERTO'	**Parcial**: atendimentos em aberto são uma fração mínima da tabela (Q1, painel de internados).
`ix_tutor_nome_lower`	tutor (lower(nome) text_pattern_ops)	**De expressão**: busca no balcão por `lower(nome) LIKE 'mar%'`, sem diferenciar maiúsculas.

`ix_atproc_procedimento`
`ix_prescricao_medicamento`
`ix_vetesp_especialidade`
`ix_procesp_especie`
`ix_medesp_especie`
	2ª coluna das associativas	A PK composta só serve buscas pela 1ª coluna. Esses índices cobrem o caminho inverso, como "onde este medicamento foi prescrito" (Q4), além de verificações de FK.

`ex_agendamento_conflito_vet`	GiST (id_veterinario, tstzrange(início, fim))	Criado pela restrição de exclusão. Além de impedir conflitos, acelera "horários do veterinário que se sobrepõem a X".

Com o volume de teste (dezenas de linhas), o planejador ainda prefere varredura sequencial, o que é o comportamento correto para tabelas pequenas. Os índices foram dimensionados para o uso real, com anos de histórico.

5.3 Outras decisões de implementação

- **Exclusão lógica.** Veterinários e procedimentos têm `ativo`; animais têm `data_obito`. Com FKs `RESTRICT`, o banco impede apagar quem tem histórico.
- **Scripts idempotentes e transacionais.** `01_ddl.sql` recria o esquema do zero e todo o DDL e DML roda entre `BEGIN` e `COMMIT`: se algo falhar, nada fica pela metade.
- **Documentação no catálogo.** Tabelas, colunas, índices, funções e visões têm `COMMENT ON`, visível no pgAdmin (aba *Properties*) e em ferramentas de documentação.
- **Nomes de restrições padronizados** (`pk_`, `fk_`, `uq_`, `ck_`, `ex_`, `ix_`), o que torna as mensagens de erro autoexplicativas.
- **Visões** `vw_animal` (animal + raça + espécie + idade + tutor) e `vw_historico_clinico` (prontuário consolidado) encapsulam junções frequentes.
- **Extensão** `btree_gist`, necessária para combinar `=` e `&&` na mesma restrição de exclusão. É uma extensão *trusted*: o dono do banco a instala sem precisar de superusuário.

5.4 Exemplo de DDL: tabela de agendamento

A tabela que concentra mais regras ilustra o estilo adotado em todo o script:

```
CREATE TABLE agendamento (
    id_agendamento      INTEGER      GENERATED BY DEFAULT AS IDENTITY,
    id_animal           INTEGER      NOT NULL,
    id_veterinario      INTEGER      NOT NULL,
    data_hora_inicio    TIMESTAMPTZ  NOT NULL,
    data_hora_fim       TIMESTAMPTZ  NOT NULL,
    tipo                VARCHAR(12)  NOT NULL DEFAULT 'CONSULTA',
    motivo              VARCHAR(200),
    status              VARCHAR(10)  NOT NULL DEFAULT 'AGENDADO',
    motivo_cancelamento VARCHAR(200),
    criado_em           TIMESTAMPTZ  NOT NULL DEFAULT now(),

    CONSTRAINT pk_agendamento PRIMARY KEY (id_agendamento),
    CONSTRAINT fk_agendamento_animal FOREIGN KEY (id_animal)
        REFERENCES animal (id_animal)          ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_agendamento_veterinario FOREIGN KEY (id_veterinario)
        REFERENCES veterinario (id_veterinario) ON UPDATE CASCADE ON DELETE RESTRICT,
    -- chave candidata usada pela FK composta de atendimento (garante que o
    -- atendimento vinculado a um agendamento seja do MESMO animal)
    CONSTRAINT uq_agendamento_animal UNIQUE (id_agendamento, id_animal),
    CONSTRAINT ck_agendamento_periodo CHECK (data_hora_fim > data_hora_inicio),
    CONSTRAINT ck_agendamento_duracao CHECK (data_hora_fim - data_hora_inicio <= INTERVAL '8 hours'),
    CONSTRAINT ck_agendamento_tipo    CHECK (tipo IN
        ('CONSULTA', 'RETORNO', 'VACINACAO', 'EXAME', 'CIRURGIA')),
    CONSTRAINT ck_agendamento_status  CHECK (status IN ('AGENDADO', 'REALIZADO', 'CANCELADO')),
    CONSTRAINT ck_agendamento_cancel  CHECK (
        (status = 'CANCELADO') = (motivo_cancelamento IS NOT NULL)),
    -- um veterinário não pode ter dois horários ativos sobrepostos
    CONSTRAINT ex_agendamento_conflito_vet EXCLUDE USING gist (
        id_veterinario WITH =,
        tstzrange(data_hora_inicio, data_hora_fim, '[)') WITH &&
    ) WHERE (status <> 'CANCELADO')
);
```

5.5 Gatilhos

Gatilho	Momento	Regra
`trg_atendimento_vincula_agendamento`	AFTER INSERT/UPDATE em atendimento	Barra atendimento de agendamento cancelado e marca o agendamento como REALIZADO (RN06).
`trg_agendamento_valida_status`	BEFORE UPDATE em agendamento	Agendamento que já gerou atendimento não pode ser cancelado nem reaberto (RN06).
`trg_atendimento_valida_animal`	BEFORE INSERT/UPDATE em atendimento	Barra atendimento com data posterior ao óbito (RN09).
`trg_atproc_valida_especie` `trg_prescricao_valida_especie`	BEFORE INSERT/UPDATE em atendimento_procedimento e prescricao	Barra procedimento ou medicamento não indicado para a espécie do animal (RN12). Uma só função atende às duas tabelas, usando `TG_TABLE_NAME`.
`trg_atproc_preco_padrao`	BEFORE INSERT em atendimento_procedimento	Se o valor não for informado, usa o preço de tabela vigente (RN08).

```
CREATE OR REPLACE FUNCTION fn_atendimento_vincula_agendamento()
RETURNS trigger
LANGUAGE plpgsql AS
$$
DECLARE
    v_status agendamento.status%TYPE;
BEGIN
    IF NEW.id_agendamento IS NULL THEN
```

```
        RETURN NEW;
    END IF;

    SELECT status INTO v_status
      FROM agendamento
     WHERE id_agendamento = NEW.id_agendamento
       FOR UPDATE;

    IF v_status = 'CANCELADO' THEN
        RAISE EXCEPTION 'Agendamento % está CANCELADO e não pode gerar atendimento.',
                        NEW.id_agendamento
              USING ERRCODE = 'check_violation';
    END IF;

    UPDATE agendamento
       SET status = 'REALIZADO'
     WHERE id_agendamento = NEW.id_agendamento
       AND status <> 'REALIZADO';

    RETURN NEW;
END;
$$;
```

5.6 Validação de CPF

A função `cpf_valido()` é `IMMUTABLE` (o resultado depende só da entrada), requisito para ser usada em `CHECK`. Ela rejeita sequências repetidas, que passariam na conta, e calcula os dois dígitos verificadores com pesos decrescentes e módulo 11:

```
CREATE OR REPLACE FUNCTION cpf_valido(p_cpf TEXT)
RETURNS BOOLEAN
LANGUAGE plpgsql
IMMUTABLE STRICT AS
$$
DECLARE
    d     INT[];
    soma  INT;
    dv    INT;
BEGIN
    -- formato: exatamente 11 dígitos
    IF p_cpf !~ '^[0-9]{11}$' THEN
        RETURN FALSE;
    END IF;

    -- sequências repetidas (000..., 111...) passam na conta, mas são inválidas
    IF p_cpf ~ '^(.)\1{10}$' THEN
        RETURN FALSE;
    END IF;

    d := string_to_array(p_cpf, NULL)::INT[];   -- '123...' -> {1,2,3,...}

    -- 1º dígito verificador: pesos 10..2 sobre os 9 primeiros dígitos
    soma := 0;
    FOR i IN 1..9 LOOP
        soma := soma + d[i] * (11 - i);
    END LOOP;
    dv := (soma * 10) % 11;
    IF dv = 10 THEN dv := 0; END IF;
    IF dv <> d[10] THEN
        RETURN FALSE;
    END IF;

    -- 2º dígito verificador: pesos 11..2 sobre os 10 primeiros dígitos
    soma := 0;
    FOR i IN 1..10 LOOP
        soma := soma + d[i] * (12 - i);
    END LOOP;
    dv := (soma * 10) % 11;
    IF dv = 10 THEN dv := 0; END IF;

    RETURN dv = d[11];
```

```
END;
$$;
```

6. Scripts DDL e DML

Arquivo	Conteúdo
`sql/00_criar_banco.sql`	Cria o banco `clinica_vet` (UTF-8) e define o fuso. Roda conectado ao banco `postgres`, pois `CREATE DATABASE` não pode estar dentro de uma transação.
`sql/01_ddl.sql`	Esquema, extensão, função de validação de CPF, 15 tabelas com todas as restrições, índices, 5 funções de gatilho, 2 visões e comentários.
`sql/02_dml.sql`	Carga de dados, ajuste das sequências, 7 exemplos de UPDATE, 3 de DELETE e 10 testes de integridade comentados.
`sql/03_consultas.sql`	As 5 consultas obrigatórias e 5 complementares.

6.1 Volume de dados

Os dados cobrem de janeiro de 2026 a janeiro de 2027: 30 atendimentos já realizados (quatro deles urgências ou emergências sem agendamento), duas internações em aberto, agendamentos futuros, cancelamentos com motivo, casos de não comparecimento, um óbito, um veterinário desligado e um procedimento descontinuado. A distribuição foi pensada para que cada consulta tenha resultado interessante: há empates no ranking, tutores com três animais, cinco espécies e pacientes de alto custo.

Os CPFs têm dígitos verificadores válidos, mas foram gerados aleatoriamente. As chaves são informadas na carga para deixar as referências legíveis, e as sequências `IDENTITY` são reposicionadas com `setval` ao final.

tabela	registros
`atendimento_procedimento`	72
`agendamento`	41
`prescricao`	33
`atendimento`	30
`animal`	20
`procedimento`	19
`medicamento`	17
`raca`	17
`tutor`	12
`procedimento_especie`	10
`veterinario_especialidade`	10
`medicamento_especie`	8
`especialidade`	7
`veterinario`	6
`especie`	5
Total	**307**

6.2 Exemplos de UPDATE

#	Situação	O que demonstra
8.1	Exame do Simba concluído: atendimento finalizado com laudo.	Transição EM_ABERTO → FINALIZADO respeitando o `CHECK` de coerência.
8.2	Registro do óbito do Frajola.	Exclusão lógica; concatenação em texto.
8.3	Cancelar os agendamentos futuros de animais com óbito.	`UPDATE ... FROM` com junção.
8.4	Marcar como cancelados (no-show) os horários passados sem atendimento.	`UPDATE` com `NOT EXISTS`.
8.5	Reajuste de 8% nas vacinas.	Atualização em massa; o histórico não muda (RN08).
8.6	Desligamento do Dr. Fábio.	Inativação em vez de exclusão (RN10).

#	Situação	O que demonstra
8.7	Tutor trocou telefone e e-mail.	Atualização localizada pela chave candidata (CPF).

6.3 Exemplos de DELETE

#	Situação	O que demonstra
9.1	Tramadol prescrito por engano no atendimento do hamster.	Remoção de item filho pela chave natural (atendimento + medicamento).
9.2	Citologia lançada sem ter sido feita.	Remoção em tabela associativa pela PK composta.
9.3	Cadastro duplicado de tutor feito no balcão.	`DELETE` protegido por `NOT EXISTS`: só apaga cadastros sem animais.

6.4 Testes de integridade

No fim de `02_dml.sql` há comandos comentados que **devem falhar**. Todos foram executados e o banco recusou cada um com a mensagem esperada:

Tentativa	Barrada por
Marcar horário sobreposto para a mesma veterinária	`ex_agendamento_conflito_vet`
Gerar atendimento de agendamento cancelado	gatilho `fn_atendimento_vincula_agendamento`
Vincular atendimento a agendamento de outro animal	FK composta `fk_atendimento_agendamento`
Atender animal após o óbito	gatilho `fn_atendimento_valida_animal`
Apagar tutor que tem animais	FK `fk_animal_tutor` (RESTRICT)
Cancelar agendamento sem motivo	`ck_agendamento_cancel`
Cancelar agendamento que já virou atendimento	gatilho `fn_agendamento_valida_status`
Cadastrar tutor com CPF de dígito verificador errado	`ck_tutor_cpf` (função `cpf_valido`)
Prescrever Simparic (só cão) para uma gata	gatilho `fn_valida_especie`
Aplicar vacina V10 (só cão) em um gato	gatilho `fn_valida_especie`

7. Consultas obrigatórias

Cada consulta é apresentada com o objetivo de negócio, o SQL exatamente como está em `03_consultas.sql`, a explicação da lógica e o resultado obtido sobre os dados de teste. Os resultados foram gerados automaticamente pelo script que monta este PDF, executando as consultas no banco em 27/10/2026. Parâmetros como datas ficam em uma CTE `params` no início da consulta, para facilitar a troca no pgAdmin.

Q1. Animais com atendimentos em aberto ou agendamentos futuros

Objetivo: painel da recepção, com quem está em atendimento ou internado agora e quem tem horário marcado, já com o telefone do tutor para confirmação.

```sql
WITH params AS (
    SELECT now() AS agora
),
em_aberto AS (
    SELECT at.id_animal,
           'Atendimento em aberto'::text           AS situacao,
           at.data_hora_inicio                     AS data_referencia,
           v.nome                                  AS veterinario,
           at.queixa_principal                     AS detalhe
      FROM atendimento at
      JOIN veterinario v ON v.id_veterinario = at.id_veterinario
     WHERE at.status = 'EM_ABERTO'
),
futuros AS (
    SELECT ag.id_animal,
           'Agendamento futuro'::text,
           ag.data_hora_inicio,
           v.nome,
           ag.motivo
      FROM agendamento ag
      JOIN veterinario v ON v.id_veterinario = ag.id_veterinario
      CROSS JOIN params p
     WHERE ag.status = 'AGENDADO'
       AND ag.data_hora_inicio > p.agora
)
SELECT a.animal,
       a.especie,
       a.tutor,
       a.telefone_tutor,
       s.situacao,
       to_char(s.data_referencia, 'DD/MM/YYYY HH24:MI') AS data_hora,
       s.veterinario,
       s.detalhe
  FROM (SELECT * FROM em_aberto
        UNION ALL
        SELECT * FROM futuros) s
  JOIN vw_animal a ON a.id_animal = s.id_animal
 ORDER BY s.situacao DESC,           -- "Atendimento em aberto" primeiro
          s.data_referencia;
```

Como funciona. Duas CTEs produzem conjuntos com as mesmas colunas: `em_aberto` (atendimentos com status `EM_ABERTO`) e `futuros` (agendamentos `AGENDADO` com início depois de `now()`). O `UNION ALL` junta os dois sem eliminar duplicatas, já que um animal pode estar internado e ter retorno marcado ao mesmo tempo, como o Max. A junção com `vw_animal` traz espécie, tutor e telefone. Os dois filtros correspondem aos índices parciais `ix_atendimento_em_aberto` e `ix_agendamento_futuro`.

animal	especie	tutor	telefone_tutor	situacao	data_hora	veterinario	detalhe
Max	Cão	Lucas Martins Ferreira	34995678905	Atendimento em aberto	26/10/2026 07:30	Dr. Bruno Henrique Costa	Abdômen distendido e ânsia improdutiva

animal	especie	tutor	telefone_tutor	situacao	data_hora	veterinario	detalhe
Lola	Cão	Gustavo Henrique Silva	34997890107	Atendimento em aberto	27/10/2026 09:00	Dra. Carolina Mendes Rocha	Respiração ofegante
Mel	Cão	Ricardo Almeida Souza	34992345602	Agendamento futuro	04/11/2026 10:00	Dra. Ana Paula Ferreira	Reavaliação da dermatite
Max	Cão	Lucas Martins Ferreira	34995678905	Agendamento futuro	09/11/2026 14:00	Dr. Bruno Henrique Costa	Retorno pós-cirúrgico (gastropexia)
Zeus	Cão	André Luiz Cardoso	34999012309	Agendamento futuro	10/11/2026 10:30	Dra. Carolina Mendes Rocha	Acompanhamento cardiológico
Amora	Coelho	Lucas Martins Ferreira	34995678905	Agendamento futuro	12/11/2026 14:00	Dr. Diego Alves Martins	Avaliação dentária
Kiara	Gato	André Luiz Cardoso	34999012309	Agendamento futuro	18/11/2026 09:00	Dr. Bruno Henrique Costa	Castração eletiva (OSH)
Nina	Cão	Fernanda Lima Rocha	34994567804	Agendamento futuro	25/11/2026 15:00	Dra. Elisa Tavares Lima	Ultrassonografia abdominal
Pipoca	Ave	Mariana Oliveira Santos	34991234501	Agendamento futuro	02/12/2026 16:00	Dr. Diego Alves Martins	Retorno comportamental
Thor	Cão	Mariana Oliveira Santos	34991234501	Agendamento futuro	14/01/2027 10:00	Dra. Ana Paula Ferreira	Vacinação anual

10 linhas retornadas.

Q2. Ranking de veterinários por quantidade de atendimentos em um período

Objetivo: medir a produtividade da equipe entre duas datas (aqui, de 01/01 a 31/10/2026).

```
WITH params AS (
    SELECT DATE '2026-01-01' AS data_inicio,
           DATE '2026-10-31' AS data_fim
),
contagem AS (
    SELECT v.id_veterinario,
           v.nome,
           v.ativo,
           count(at.id_atendimento)                                          AS atendimentos,
           count(at.id_atendimento) FILTER (WHERE at.carater <> 'ELETIVO')   AS urgencias_emergencias,
           count(DISTINCT at.id_animal)                                      AS pacientes_distintos
      FROM veterinario v
      CROSS JOIN params p
      LEFT JOIN atendimento at
             ON at.id_veterinario    = v.id_veterinario
            AND at.data_hora_inicio >= p.data_inicio
            AND at.data_hora_inicio <  p.data_fim + 1      -- inclui o último dia inteiro
     GROUP BY v.id_veterinario, v.nome, v.ativo
)
SELECT RANK() OVER (ORDER BY atendimentos DESC)                              AS posicao,
       nome                                                                  AS veterinario,
       CASE WHEN ativo THEN 'Ativo' ELSE 'Desligado' END                     AS situacao,
       atendimentos,
       urgencias_emergencias,
       pacientes_distintos,
       round(100.0 * atendimentos / NULLIF(SUM(atendimentos) OVER (), 0), 1) AS percentual
  FROM contagem
 ORDER BY posicao, veterinario;
```

Como funciona. O período fica na CTE `params`. O ponto principal é o `LEFT JOIN` com o filtro de datas **na cláusula `ON`**: se o filtro estivesse no `WHERE`, veterinários sem atendimentos no período sumiriam do relatório. Por isso se conta `count(at.id_atendimento)`, que ignora nulos, e não `count(*)`. O `FILTER` conta urgências e emergências na mesma passada. Na consulta externa, `RANK()` numera as posições e repete a posição em empates (dois ve-

terinários em 3º lugar, e o seguinte vai para 5º), e `SUM() OVER ()` calcula o total geral para o percentual sem uma segunda consulta. O limite superior usa `< data_fim + 1` para incluir o último dia inteiro, já que a coluna tem hora.

posicao	veterinario	situacao	atendimentos	urgencias_emergencias	pacientes_distintos	percentual
1	Dra. Ana Paula Ferreira	Ativo	10	1	9	33,3
2	Dra. Carolina Mendes Rocha	Ativo	7	1	3	23,3
3	Dr. Bruno Henrique Costa	Ativo	4	2	3	13,3
3	Dr. Diego Alves Martins	Ativo	4	0	4	13,3
5	Dra. Elisa Tavares Lima	Ativo	3	0	3	10,0
6	Dr. Fábio Nogueira Prado	Desligado	2	0	1	6,7

6 linhas retornadas.

Q3. Tutores com mais de um animal e histórico resumido de procedimentos

Objetivo: identificar os clientes com mais vínculo (candidatos a plano família) e ver o que já fizeram na clínica.

```
WITH tutores_multi AS (
    SELECT id_tutor
      FROM animal
     GROUP BY id_tutor
    HAVING count(*) > 1
),
animais AS (
    SELECT a.id_tutor,
           string_agg(a.nome || ' (' || e.nome || ')', ', ' ORDER BY a.nome) AS animais,
           count(*)                                                          AS qtd_animais
      FROM animal a
      JOIN raca    r ON r.id_raca    = a.id_raca
      JOIN especie e ON e.id_especie = r.id_especie
     WHERE a.id_tutor IN (SELECT id_tutor FROM tutores_multi)
     GROUP BY a.id_tutor
),
atend AS (
    SELECT a.id_tutor,
           count(at.id_atendimento)  AS atendimentos,
           max(at.data_hora_inicio)  AS ultimo_atendimento
      FROM animal a
      LEFT JOIN atendimento at ON at.id_animal = a.id_animal
     WHERE a.id_tutor IN (SELECT id_tutor FROM tutores_multi)
     GROUP BY a.id_tutor
),
proc AS (
    SELECT a.id_tutor,
           p.nome,
           SUM(ap.quantidade)                    AS qtd,
           SUM(ap.quantidade * ap.valor_unitario) AS valor
      FROM animal a
      JOIN atendimento              at ON at.id_animal       = a.id_animal
      JOIN atendimento_procedimento ap ON ap.id_atendimento  = at.id_atendimento
      JOIN procedimento             p  ON p.id_procedimento  = ap.id_procedimento
     WHERE a.id_tutor IN (SELECT id_tutor FROM tutores_multi)
     GROUP BY a.id_tutor, p.nome
)
SELECT t.nome                                                    AS tutor,
       an.qtd_animais,
       an.animais,
       ate.atendimentos,
       to_char(ate.ultimo_atendimento, 'DD/MM/YYYY') AS ultimo_atendimento,
       COALESCE(SUM(pr.valor), 0)                                AS valor_total,
       string_agg(pr.nome || ' ×' || pr.qtd, '; ' ORDER BY pr.qtd DESC, pr.nome) AS procedimentos
  FROM tutor t
  JOIN animais an  ON an.id_tutor  = t.id_tutor
  JOIN atend   ate ON ate.id_tutor = t.id_tutor
  LEFT JOIN proc pr ON pr.id_tutor = t.id_tutor
```

```
GROUP BY t.id_tutor, t.nome, an.qtd_animais, an.animais, ate.atendimentos, ate.ultimo_atendimento
ORDER BY an.qtd_animais DESC, valor_total DESC;
```

Como funciona. A CTE `tutores_multi` seleciona os tutores com `HAVING count(*) > 1`. As outras três CTEs agregam, cada uma em seu próprio nível, os animais do tutor (`string_agg` ordenado), os atendimentos (com `LEFT JOIN`, para que um animal nunca atendido não suma) e os procedimentos com quantidade e valor. Agregar em CTEs separadas antes de juntar evita o problema da **multiplicação de linhas**: se tudo fosse juntado de uma vez, cada atendimento seria repetido para cada procedimento e as contagens sairiam infladas. A consulta final monta o resumo "Procedimento ×quantidade", do mais frequente para o menos frequente.

tutor	qtd_animais	animais	atendimentos	ultimo_atendimento	valor_total	procedimentos
Mariana Oliveira Santos	3	Luna (Gato), Pipoca (Ave), Thor (Cão)	4	01/07/2026	1.615,00	Consulta clínica geral ×2; Vacina antirrábica ×2; Consulta especializada ×1; Fluidoterapia ×1; Hemograma completo ×1; Perfil bioquímico (renal e hepático) ×1; Profilaxia dentária (tartarectomia) ×1; Vacina polivalente canina (V10) ×1; Vacina quádrupla felina (V4) ×1
Lucas Martins Ferreira	2	Amora (Coelho), Max (Cão)	3	26/10/2026	4.230,00	Radiografia (2 incidências) ×3; Consulta clínica geral ×2; Fluidoterapia ×2; Consulta especializada ×1; Diária de internação ×1; Gastropexia (correção de torção gástrica) ×1
Fernanda Lima Rocha	2	Frajola (Gato), Nina (Cão)	5	25/08/2026	2.510,00	Fluidoterapia ×4; Hemograma completo ×3; Perfil bioquímico (renal e hepático) ×3; Diária de internação ×2; Castração (OSH/orquiectomia) ×1; Consulta clínica geral ×1; Consulta de retorno ×1; Radiografia (2 incidências) ×1; Urinálise ×1
Gustavo Henrique Silva	2	Lola (Cão), Mingau (Gato)	3	27/10/2026	1.210,00	Consulta clínica geral ×3; Radiografia (2 incidências) ×2; Diária de internação ×1; Vacina antirrábica ×1; Vacina quádrupla felina (V4) ×1
André Luiz Cardoso	2	Kiara (Gato), Zeus (Cão)	3	16/09/2026	920,00	Eletrocardiograma ×2; Consulta de retorno ×1; Consulta especializada ×1; Radiografia (2 incidências) ×1; Ultrassonografia abdominal ×1
Ricardo Almeida Souza	2	Mel (Cão), Simba (Gato)	4	08/10/2026	685,00	Citologia (pele/ouvido) ×1; Consulta clínica geral ×1; Consulta de retorno ×1; Consulta especializada ×1; Hemograma completo ×1; Ultrassonografia abdominal ×1
Helena Barbosa Nunes	2	Belinha (Cão), Tom (Gato)	2	29/07/2026	560,00	Consulta clínica geral ×2; Citologia (pele/ouvido) ×1; Vacina antirrábica ×1; Vacina quádrupla felina (V4) ×1

7 linhas retornadas.

Q4. Medicamentos mais prescritos por espécie de animal

Objetivo: apoiar o planejamento de estoque da farmácia e mostrar o perfil terapêutico de cada espécie.

```
WITH contagem AS (
    SELECT e.nome                          AS especie,
           m.nome_comercial || ' ' || m.concentracao AS medicamento,
           m.principio_ativo,
           count(*)                        AS prescricoes,
           count(DISTINCT an.id_animal)    AS animais_distintos
      FROM prescricao  pr
      JOIN medicamento m  ON m.id_medicamento = pr.id_medicamento
      JOIN atendimento at ON at.id_atendimento = pr.id_atendimento
      JOIN animal      an ON an.id_animal     = at.id_animal
```

```
      JOIN raca        r  ON r.id_raca        = an.id_raca
      JOIN especie     e  ON e.id_especie     = r.id_especie
     GROUP BY e.nome, m.id_medicamento, m.nome_comercial, m.concentracao, m.principio_ativo
),
ranking AS (
    SELECT c.*,
           DENSE_RANK() OVER (PARTITION BY especie ORDER BY prescricoes DESC) AS posicao
      FROM contagem c
)
SELECT especie, posicao, medicamento, principio_ativo, prescricoes, animais_distintos
  FROM ranking
 WHERE posicao <= 3
 ORDER BY especie, posicao, medicamento;
```

Como funciona. A espécie não está na prescrição: ela é alcançada pela cadeia prescrição → atendimento → animal → raça → espécie (seis tabelas). A CTE `contagem` agrupa por espécie e medicamento e conta prescrições e animais distintos. Animais distintos é relevante porque um animal crônico pode receber o mesmo remédio várias vezes, como o Zeus com o Vetmedin. Em seguida, `DENSE_RANK() OVER (PARTITION BY especie ...)` reinicia a numeração a cada espécie. Foi usado `DENSE_RANK` em vez de `ROW_NUMBER` para que medicamentos empatados não sejam descartados arbitrariamente, e o filtro `posicao <= 3` traz o "top 3" de cada espécie.

especie	posicao	medicamento	principio_ativo	prescricoes	animais_distintos
Ave	1	Enrofloxacino 2,5% 25 mg/mL	enrofloxacino	1	1
Coelho	1	Maxicam 2 mg	meloxicam	1	1
Cão	1	Agemoxi CL 250 mg	amoxicilina + clavulanato de potássio	3	3
Cão	1	Maxicam 2 mg	meloxicam	3	3
Cão	1	Tramadol 50 mg	cloridrato de tramadol	3	2
Cão	1	Vetmedin 1,25 mg	pimobendan	3	2
Cão	2	Furosemida 40 mg	furosemida	2	1
Cão	2	Ondansetrona 2 mg/mL	cloridrato de ondansetrona	2	2
Cão	2	Prediderm 20 mg	prednisolona	2	1
Cão	3	Apoquel 5,4 mg	oclacitinib	1	1
Cão	3	Drontal Plus 10 kg	praziquantel + pirantel + febantel	1	1
Cão	3	Gabapentina 100 mg	gabapentina	1	1
Cão	3	Omeprazol 10 mg	omeprazol	1	1
Cão	3	Otomax 15 g	gentamicina + betametasona + clotrimazol	1	1
Cão	3	Simparic 40 mg	sarolaner	1	1
Gato	1	Revolution 6% (45 mg)	selamectina	3	3
Gato	2	Omeprazol 10 mg	omeprazol	1	1
Gato	2	Ondansetrona 2 mg/mL	cloridrato de ondansetrona	1	1
Gato	2	Tobramicina 3 mg/mL	tobramicina	1	1
Roedor	1	Vitamina C 200 mg/mL	ácido ascórbico	1	1

20 linhas retornadas.

Q5. Junção de 3+ tabelas, agregação e subconsulta

Objetivo: "quais pacientes gastaram acima da média?", para acompanhar casos de alto custo (doenças crônicas, cirurgias) e oferecer planos de saúde pet.

```
SELECT an.nome                                          AS animal,
       e.nome                                           AS especie,
       t.nome                                           AS tutor,
       count(DISTINCT at.id_atendimento)                AS atendimentos,
       SUM(ap.quantidade * ap.valor_unitario)           AS gasto_total,
```

```sql
       round(m.media, 2)                                AS media_geral,
       round(100.0 * (SUM(ap.quantidade * ap.valor_unitario) / m.media - 1), 1) AS acima_da_media_pct
  FROM animal an
  JOIN raca                    r  ON r.id_raca        = an.id_raca
  JOIN especie                 e  ON e.id_especie     = r.id_especie
  JOIN tutor                   t  ON t.id_tutor       = an.id_tutor
  JOIN atendimento             at ON at.id_animal     = an.id_animal
  JOIN atendimento_procedimento ap ON ap.id_atendimento = at.id_atendimento
  CROSS JOIN (
        -- subconsulta: gasto médio por paciente atendido
        SELECT avg(gasto) AS media
          FROM (SELECT at2.id_animal,
                       SUM(ap2.quantidade * ap2.valor_unitario) AS gasto
                  FROM atendimento at2
                  JOIN atendimento_procedimento ap2 ON ap2.id_atendimento = at2.id_atendimento
                 GROUP BY at2.id_animal) gasto_por_animal
       ) m
 GROUP BY an.id_animal, an.nome, e.nome, t.nome, m.media
HAVING SUM(ap.quantidade * ap.valor_unitario) > m.media
 ORDER BY gasto_total DESC;
```

Como funciona. A consulta junta **seis tabelas** (animal, raça, espécie, tutor, atendimento, atendimento_procedimento) e agrega o gasto por animal com `SUM(quantidade × valor_unitario)`, usando o valor cobrado e não o preço de tabela. A **subconsulta**, em uma tabela derivada aninhada, primeiro calcula o gasto de cada paciente e depois a média desses gastos. A média por paciente é diferente da média por item, que seria distorcida por atendimentos com muitos itens. Como a subconsulta devolve uma única linha, o `CROSS JOIN` a disponibiliza tanto no `HAVING`, que filtra quem está acima, quanto no `SELECT`, que mostra quanto acima está, em %. Atende aos três requisitos: junção de três ou mais tabelas, agregação e subconsulta.

animal	especie	tutor	atendimentos	gasto_total	media_geral	acima_da_media_pct
Max	Cão	Lucas Martins Ferreira	2	3.850,00	666,00	478,1
Frajola	Gato	Fernanda Lima Rocha	3	1.545,00	666,00	132,0
Thor	Cão	Mariana Oliveira Santos	2	1.055,00	666,00	58,4
Nina	Cão	Fernanda Lima Rocha	2	965,00	666,00	44,9
Lola	Cão	Gustavo Henrique Silva	2	870,00	666,00	30,6
Zeus	Cão	André Luiz Cardoso	2	740,00	666,00	11,1

6 linhas retornadas.

8. Consultas complementares

Cinco consultas extras, também em `03_consultas.sql`, cobrem necessidades comuns da clínica e exercitam outros recursos do SQL.

C1. Prontuário de um animal

Usa a visão `vw_historico_clinico`, que consolida procedimentos e prescrições de cada atendimento com `LEFT JOIN LATERAL` e `string_agg`. Os atendimentos sem itens continuam aparecendo. A consulta só filtra pelo animal informado em `params`.

```
WITH params AS (SELECT 'Zeus'::varchar AS animal)
SELECT to_char(h.data_hora_inicio, 'DD/MM/YYYY') AS data,
       h.veterinario,
       h.queixa_principal,
       h.diagnostico,
       h.peso_kg,
       h.procedimentos,
       h.prescricoes,
       h.valor_total
  FROM vw_historico_clinico h
  JOIN params p ON h.animal = p.animal
 ORDER BY h.data_hora_inicio DESC;
```

data	veterinario	queixa_principal	diagnostico	peso_kg	procedimentos	prescricoes	valor_total
16/09/2026	Dra. Carolina Mendes Rocha	Acompanhamento cardiológico	Doença valvar mitral compensada.	37,9	Consulta de retorno; Eletrocardiograma	Furosemida 40 mg — 1 comprimido a cada 12 h; Vetmedin 1,25 mg — 1,5 comprimidos a cada 12 h	220,00
27/03/2026	Dra. Carolina Mendes Rocha	Tosse e intolerância ao exercício	Doença valvar mitral (ACVIM B2).	38,4	Consulta especializada; Eletrocardiograma; Radiografia (2 incidências)	Furosemida 40 mg — 1 comprimido a cada 12 h por 30 dia(s); Vetmedin 1,25 mg — 1 comprimido a cada 12 h	520,00

2 linhas retornadas.

C2. Faturamento mensal por categoria

Transforma linhas em colunas (*pivot*) com `SUM(...) FILTER (WHERE ...)`, uma coluna por categoria, agrupando por `date_trunc('month', ...)`. O pico de outubro corresponde à gastropexia de emergência do Max.

```
SELECT to_char(date_trunc('month', at.data_hora_inicio), 'YYYY-MM')                    AS mes,
       SUM(ap.quantidade * ap.valor_unitario) FILTER (WHERE p.categoria = 'CONSULTA')   AS consultas,
       SUM(ap.quantidade * ap.valor_unitario) FILTER (WHERE p.categoria IN ('EXAME', 'IMAGEM')) AS exames,
       SUM(ap.quantidade * ap.valor_unitario) FILTER (WHERE p.categoria = 'VACINA')     AS vacinas,
       SUM(ap.quantidade * ap.valor_unitario) FILTER (WHERE p.categoria = 'CIRURGIA')   AS cirurgias,
       SUM(ap.quantidade * ap.valor_unitario) FILTER (WHERE p.categoria IN ('TERAPIA', 'INTERNACAO')) AS
terapia_internacao,
       SUM(ap.quantidade * ap.valor_unitario)                                          AS total
  FROM atendimento at
  JOIN atendimento_procedimento ap ON ap.id_atendimento  = at.id_atendimento
  JOIN procedimento             p  ON p.id_procedimento  = ap.id_procedimento
```

```
  WHERE at.data_hora_inicio >= DATE '2026-01-01'
    AND at.data_hora_inicio <  DATE '2027-01-01'
  GROUP BY 1
  ORDER BY 1;
```

mes	consultas	exames	vacinas	cirurgias	terapia_internacao	total
2026-01	520,00	255,00	180,00	—	—	955,00
2026-02	520,00	400,00	—	—	—	920,00
2026-03	520,00	365,00	190,00	650,00	90,00	1.815,00
2026-04	370,00	225,00	—	—	90,00	685,00
2026-05	380,00	280,00	190,00	—	90,00	940,00
2026-06	300,00	320,00	—	—	—	620,00
2026-07	300,00	255,00	190,00	450,00	90,00	1.285,00
2026-08	—	345,00	—	—	680,00	1.025,00
2026-09	520,00	210,00	—	—	—	730,00
2026-10	300,00	565,00	—	2.800,00	680,00	4.345,00

10 linhas retornadas.

C3. Cães e gatos sem vacina nos últimos 12 meses

Lista de contatos para campanha de vacinação. O anti-join `NOT EXISTS` encontra quem *não* tem vacina recente, uma subconsulta escalar correlacionada mostra a data da última vacina (vazia quando nunca vacinou) e o filtro `vivo` exclui o paciente que veio a óbito.

```
SELECT a.animal,
       a.especie,
       a.tutor,
       a.telefone_tutor,
       (SELECT to_char(max(at.data_hora_inicio), 'DD/MM/YYYY')
          FROM atendimento at
          JOIN atendimento_procedimento ap ON ap.id_atendimento = at.id_atendimento
          JOIN procedimento p              ON p.id_procedimento = ap.id_procedimento
         WHERE at.id_animal = a.id_animal
           AND p.categoria  = 'VACINA')                        AS ultima_vacina
  FROM vw_animal a
 WHERE a.vivo
   AND a.especie IN ('Cão', 'Gato')
   AND NOT EXISTS (
        SELECT 1
          FROM atendimento at
          JOIN atendimento_procedimento ap ON ap.id_atendimento = at.id_atendimento
          JOIN procedimento p              ON p.id_procedimento = ap.id_procedimento
         WHERE at.id_animal        = a.id_animal
           AND p.categoria         = 'VACINA'
           AND at.data_hora_inicio >= now() - INTERVAL '12 months')
 ORDER BY a.especie, a.animal;
```

animal	especie	tutor	telefone_tutor	ultima_vacina
Belinha	Cão	Helena Barbosa Nunes	34992233411	—
Bidu	Cão	José Carlos Pereira	34988776655	—
Bolt	Cão	Patrícia Gomes Ribeiro	34996789006	—
Lola	Cão	Gustavo Henrique Silva	34997890107	—
Max	Cão	Lucas Martins Ferreira	34995678905	—
Mel	Cão	Ricardo Almeida Souza	34992345602	—
Nina	Cão	Fernanda Lima Rocha	34994567804	—
Zeus	Cão	André Luiz Cardoso	34999012309	—

animal	especie	tutor	telefone_tutor	ultima_vacina
Kiara	Gato	André Luiz Cardoso	34999012309	—
Simba	Gato	Ricardo Almeida Souza	34992345602	—

10 linhas retornadas.

C4. Taxa de cancelamento por veterinário

Distribuição dos agendamentos por status em uma única passada sobre a tabela, usando agregações condicionais com `FILTER`.

```
SELECT v.nome                                                    AS veterinario,
       count(*)                                                  AS agendamentos,
       count(*) FILTER (WHERE ag.status = 'REALIZADO')           AS realizados,
       count(*) FILTER (WHERE ag.status = 'CANCELADO')           AS cancelados,
       count(*) FILTER (WHERE ag.status = 'AGENDADO')            AS pendentes,
       round(100.0 * count(*) FILTER (WHERE ag.status = 'CANCELADO') / count(*), 1) AS taxa_cancelamento_pct
  FROM agendamento ag
  JOIN veterinario v ON v.id_veterinario = ag.id_veterinario
 GROUP BY v.id_veterinario, v.nome
 ORDER BY taxa_cancelamento_pct DESC, agendamentos DESC;
```

veterinario	agendamentos	realizados	cancelados	pendentes	taxa_cancelamento_pct
Dra. Ana Paula Ferreira	15	9	4	2	26,7
Dr. Bruno Henrique Costa	5	2	1	2	20,0
Dr. Diego Alves Martins	7	4	1	2	14,3
Dra. Carolina Mendes Rocha	8	6	1	1	12,5
Dra. Elisa Tavares Lima	4	3	0	1	0,0
Dr. Fábio Nogueira Prado	2	2	0	0	0,0

6 linhas retornadas.

C5. Itens restritos por espécie

Mostra, para cada espécie, os procedimentos e medicamentos que têm restrição e a incluem. Usa dois `LEFT JOIN` independentes e `string_agg(DISTINCT ...)` para não repetir itens, já que as duas junções se multiplicam.

```
SELECT e.nome                                                    AS especie,
       string_agg(DISTINCT p.nome, '; ')                         AS procedimentos_restritos,
       string_agg(DISTINCT m.nome_comercial || ' ' || m.concentracao, '; ') AS medicamentos_restritos
  FROM especie e
  LEFT JOIN procedimento_especie pe ON pe.id_especie      = e.id_especie
  LEFT JOIN procedimento         p  ON p.id_procedimento  = pe.id_procedimento
  LEFT JOIN medicamento_especie  me ON me.id_especie      = e.id_especie
  LEFT JOIN medicamento          m  ON m.id_medicamento   = me.id_medicamento
 GROUP BY e.id_especie, e.nome
 ORDER BY e.id_especie;
```

especie	procedimentos_restritos	medicamentos_restritos
Cão	Castração (OSH/orquiectomia); Gastropexia (correção de torção gástrica); Profilaxia dentária (tartarectomia); Vacina antirrábica; Vacina polivalente canina (V10)	Apoquel 5,4 mg; Drontal Plus 10 kg; Revolution 6% (45 mg); Simparic 40 mg; Vetmedin 1,25 mg
Gato	Castração (OSH/orquiectomia); Profilaxia dentária (tartarectomia); Vacina antirrábica; Vacina quádrupla felina (V4)	Revolution 6% (45 mg)
Ave	—	—
Roedor	—	Vitamina C 200 mg/mL
Coelho	Castração (OSH/orquiectomia)	Vitamina C 200 mg/mL

5 linhas retornadas.

9. Interface web

Para demonstrar o banco em uso, foi feita uma interface web em **Node.js** (Express e o driver `pg`), com front-end em HTML, CSS e JavaScript sem framework. Ela fica na pasta `app/` e roda com `npm install` e `npm start`. As telas usam as mesmas visões e consultas deste relatório.

A interface não reimplementa as regras de negócio: ela envia os comandos ao PostgreSQL e, quando o banco recusa (CPF inválido, conflito de agenda, motivo de cancelamento ausente), mostra a mensagem correspondente na tela. Assim, as garantias valem tanto para a interface quanto para o pgAdmin.

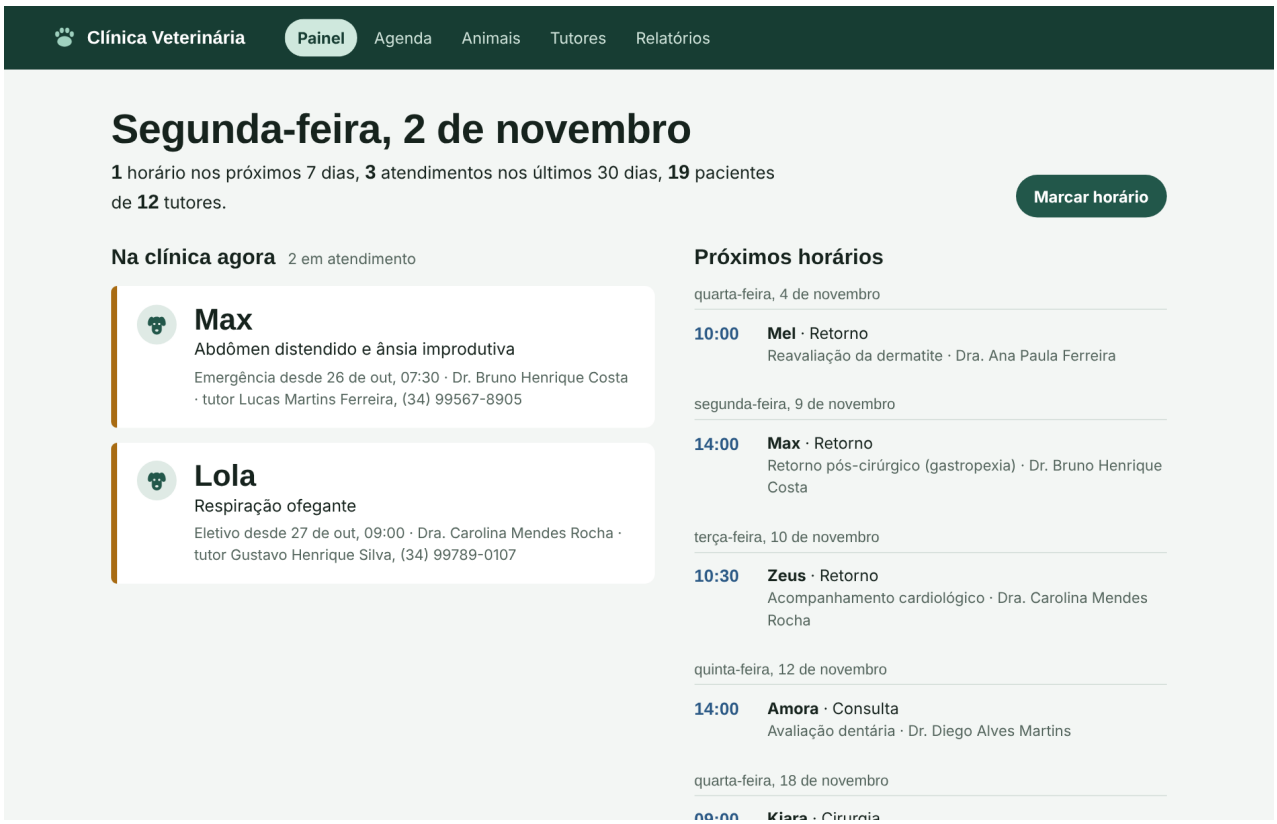


Figura 3. Painel: pacientes em atendimento (consulta Q1) e próximos horários agrupados por dia.

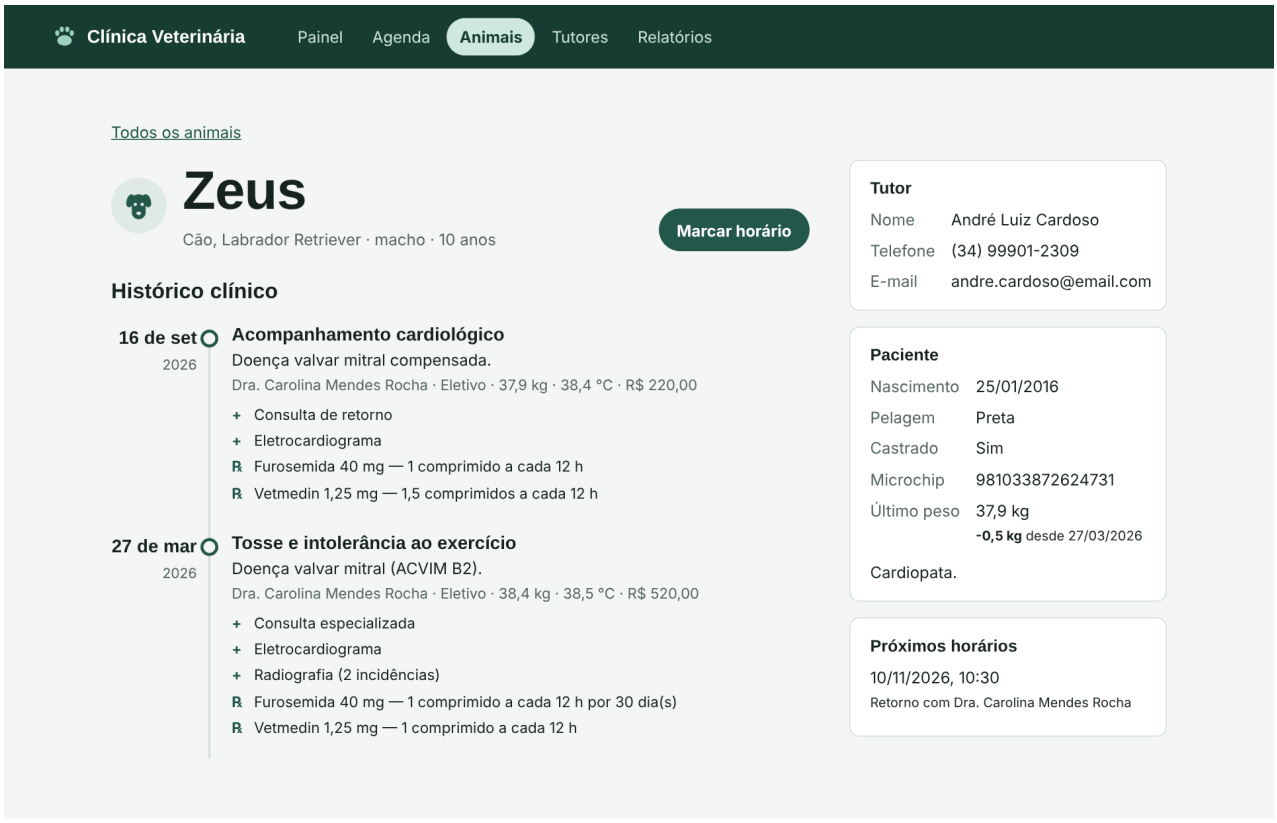


Figura 4. Prontuário montado a partir da visão `vw_historico_clinico`, com procedimentos (+), prescrições (℞) e variação de peso.

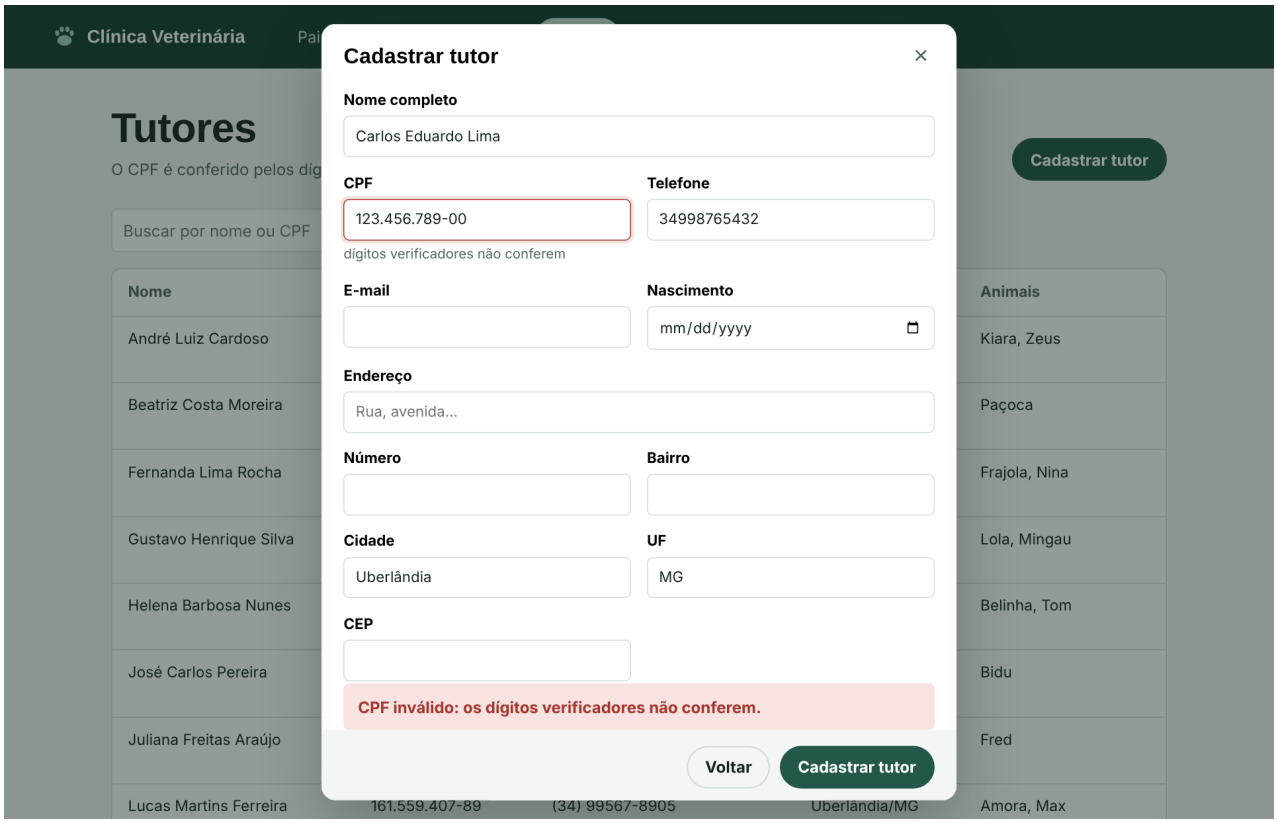


Figura 5. Cadastro de tutor recusado pelo banco por CPF inválido (restrição `ck_tutor_cpf`).

10. Considerações finais

O banco implementado cobre todas as entidades e regras pedidas e leva para dentro do SGBD as regras que, em muitos sistemas, ficam só na aplicação: conflito de agenda, vínculo coerente entre agendamento e atendimento, preço histórico e bloqueio pós-óbito. Assim, qualquer cliente que acesse o banco, seja o pgAdmin, uma API ou um script, encontra as mesmas garantias. O esquema está em 3FN (e FNBC), com uma única redundância, documentada e mantida por gatilho.

Como evoluções naturais, ficam: controle de estoque de medicamentos (com baixa automática a partir das prescrições), um módulo financeiro de pagamentos, uma carteira de vacinação com datas de reforço e perfis de acesso (papéis `recepcao`, `veterinario`, `gestor`) com `GRANT` por esquema e segurança em nível de linha.

Apêndice A — Como executar

No pgAdmin 4

1. Conecte-se ao servidor, clique com o botão direito no banco `postgres`, abra *Query Tool* e carregue `00_criar_banco.sql`. Selecione só o `CREATE DATABASE` e execute (F5); depois selecione o restante e execute de novo. O pgAdmin envia o texto todo de uma vez, e `CREATE DATABASE` não pode rodar junto com outros comandos.
2. Atualize a árvore (F5 no painel lateral), selecione o novo banco `clinica_vet` e abra uma *Query Tool* nele.
3. Execute, nesta ordem, `01_ddl.sql`, `02_dml.sql` e `03_consultas.sql`. No último, selecione uma consulta por vez e execute com F5 para ver cada resultado na grade.
4. Os objetos ficam em *clinica_vet → Schemas → clinica*. Para consultar sem prefixar o esquema, execute `SET search_path TO clinica;`.

Pelo terminal (psql)

```
psql -U postgres -f sql/00_criar_banco.sql
psql -U postgres -d clinica_vet -f sql/01_ddl.sql
psql -U postgres -d clinica_vet -f sql/02_dml.sql
psql -U postgres -d clinica_vet -f sql/03_consultas.sql
```

Ou, com Docker, `docker compose up` sobe um PostgreSQL 16 já com o esquema e os dados carregados (ver `README.md`).

As consultas Q1 e C3 usam `now()`. Os dados foram construídos com 27/10/2026 como "hoje"; se forem executadas muito depois dessa data, os agendamentos futuros deixarão de ser futuros. Basta trocar `now()` por uma data fixa na CTE `params`.